#6

# Encode and Decode Strings

Arrays & Hashing | LeetCode 271 | NeetCode "String Encode and Decode"

## PROBLEM

Design an algorithm to encode a list of strings into a single string, and to decode that single string back into the original list of strings. The encoding must work for any possible string, including empty strings and strings that contain any character (letters, digits, spaces, punctuation — even the delimiter characters you choose to use).

```
strs = ["neet","code","love","you"] → encode →
"4#neet4#code4#love3#you" → decode → ["neet","code","love","you"]
```

```
strs = ["we","say",":","yes"] → encode → "2#we3#say1#:3#yes" →
decode → ["we","say",":","yes"]
```

## CORE IDEA

Goal

Pack a list of arbitrary strings into one string, and be able to split that one string back into the exact original list — even if the strings themselves contain commas, spaces, or any delimiter you might otherwise think to use.

Key insight

A naive delimiter (comma, space, a rare symbol) breaks the moment a string contains that exact character. The fix is to stop relying on the delimiter to mark the *end* of a string and instead have each string announce its own length up front — "4#neet" says "read exactly 4 characters starting right after this '#'", so whatever those 4 characters are (including more '#'s) can never be misread as a boundary.

> Trigger: “Serialize / deserialize a list of strings so any string is possible” → length-prefix each string instead of trusting a delimiter (Hash Map pattern's cousin: self-describing chunks).

## OPTIMAL SOLUTION

Data structure — Plain string manipulation — no extra data structure needed, just a length-prefix / delimiter convention

1. **Encode:** for each string `s` in `strs`, append `str(len(s)) + "#" + s` to the result — the string's own length, a single '#' separator, then the string itself.
2. Concatenating these length-prefixed chunks for every string in the list produces one self-describing encoded string; return it.
3. **Decode:** walk the encoded string with a pointer `i` starting at 0.
4. From `i`, scan forward with a second pointer `j` until `s[j] == '#'` — everything from `i` to `j` is the length prefix; parse it with `int(s[i:j])`.
5. Move past the '#' (`j += 1`), then slice out exactly `length` characters: `s[j : j+length]` — that's the next original string, regardless of what characters it contains.
6. Advance `i` to `j + length` (just past the string just read) and repeat until `i` reaches the end of the encoded string.

| O(n) for both encode and decode, where n is the total number of characters across all strings | O(n) for the encoded string / output list |
|---|---|
| TIME | SPACE |

## CODE

```python
class Solution:


    def encode(self, strs: List[str]) -> str:
        res = ""
        for s in strs:
            res += (str(len(s)) + "#" + s)
        return res



    def decode(self, s: str) -> List[str]:
        result = []
        i = 0
        while i < len(s):
            j=i
            while s[j] != '#':
                j += 1
            length = int(s[i:j])
            j += 1
            result.append(s[j:j+ length])
            i = j + length
        return result
```

## TIPS TO REMEMBER

Mental shortcut — “Say how long you are before you speak — then nobody needs to guess where you stop.”

Common / related — Group Anagrams, Longest Substring Without Repeating Characters, Valid Anagram

> *"I'm avoiding a fixed delimiter entirely by length-prefixing each string — since the problem allows any character in the input, no delimiter is actually safe, but a length prefix lets the decoder read exactly the right number of characters regardless of what they are, in O(n) time with two pointers."*

## EDGE CASES

- A string that's itself empty "" → encodes as "0#", decodes back to "" (length 0 slice) correctly
- An empty input list [] → encodes to "" (empty string), decodes back to []
- Very long strings — the length prefix can be multiple digits (e.g. "123#..."), and `int(s[i:j])` parses any number of digits correctly since it stops exactly at the '#'
- Strings that contain the delimiter character '#' or digits — e.g. "a#b" or "12345" → handled correctly because the decoder only reads '#' as a delimiter when it's scanning for the length prefix, never while slicing the known-length content
- A list with a single string → one length-prefixed chunk, decoded in one loop iteration